Name: - SHUBHAM DAS Date: - 27/09/2026
S.A.P I. D: - 590033500 Subject: - Programming In C

Experiment#3: -
Purpose of the experiment: - To demonstrate the proper use of editors and debugging tools along with observing and understanding different types of bugs and errors that naturally appear in C programming language while coding and compilation.

Environment: -
1. Kali Linux VM
2. Oracle VirtualBox
3. C Programming Language

Aim of the experiment: - Proper demonstration and application of editing and debugging tools as we try to understand, observe and fix different types of bugs and errors that appear naturally while compiling and executing programmes via the use of C programming language.

Glossary: -
1. Error: - Mistake made by the programmer while compiling the code of the programme. E. g Instead of 'a*b', programmer cast it as 'a+b'.

2. Bug: - Problem that came to exist in the programme because the earlier mistake made by the programmer. E. g The given output will result in a sum function, not in a multiplication function as the programmer originally desired.

3. Debugging:- Finding out the appropriate bug causing obstruction in the compilation of the programme and correcting it accordingly. E. g A function meant to add two numbers, but it treats them as text instead, so we appropriately rectify the function.

4. Compile-time Error: - These errors are safety nets provided by the programming language. The compiler scans your source code and ensures it follows all structural laws. If it finds a mistake, it refuses to generate an executable file. E. g Missing semicolons, unmatched parentheses, using an undeclared variable, or passing a string into a function that requires an integer.

5. Runtime Errors: - Your code is syntactically perfect, so the compiler passes it. However, once the programme starts running, it encounters a situation it can not handle, forcing it to crash to protect the operating system. E. g Dividing a number by zero, attempting to open a file that does not exist in the hard drive, or accessing an index outside the array's boundary.

6. Logical Error:- These are completely invisible to the computer. The syntax is flawless, and no forbidden hardware actions occur. The programme executes perfectly from the start to the

finish, but the human developer programmed the wrong instructions. E. g Using a '+' instead of a '-' sign, writing '>' instead of '<', or putting a semicolon at the end of a 'for' loop statement.

7. Syntax Errors: - Every programming language has strict grammatical rules. If you violate them, the compiler can not parse your code.
E. g

```c
#include <stdio.h>
int main() {
    int a = 10
    printf("%d\n", a);
    return 0;
};
```

```
Main.c: In function 'main':
Main.c:4:5: error: expected ',' or ';' before 'printf'
    4 |     printf("%d\n", a);
      |     ^~~~~~
```

Here, the Syntax Error is missing a semicolon (;) at the end of the third line.

8. Linker Errors: - In larger programmes, code is split across multiple files. The compiler compiles each file individually into the “object code”. The linker is the utility that glues all these seperate files together. If file A calls a function that is supposed to be inside file B, but file B does not exist or does not have that function, the linker fails.
E. g

```c
#include <stdio.h>
void calculateTotal(int price);

int main() {

    CalculateTotal(100);

    return 0;

}
```

```
Main.c: In function 'main':
Main.c:6:5: warning: implicit declaration of function 'CalculateTotal'; did
you mean 'calculateTotal'? [-Wimplicit-function-declaration]
    6 |     CalculateTotal(100);
      |     ^~~~~~~~~~~~~~
      |     calculateTotal
/usr/bin/ld: /tmp/ccCnIji2.o: in function `main':
Main.c:(.text+0xc): undefined reference to `CalculateTotal'
collect2: error: ld returned 1 exit status
```

Here, the actual body of ‘calculateTotal()’ is never defined anywhere.

---

1. Syntax/ Compilation Error: -
   E. g

```
┌──(kali㉿kali)-[~]
└─$ vi error1.c
```

```c
#include <stdio.h>
int main() {
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

   $cc error1.c -o error1
   Error1. C:5:5: error; expected ‘,’ or ‘;’ before ‘printf’
   Identify filename - error1.c, line number-5, types of message: error, remaining text - Actual Message.

   => Reading Compiler Warnings And Errors
   Do not ignore compiler messages.
   A warning does not always stop complications, but it can indicate a possible problem.

E. g

```
//sample1.c
#include <stdio.h>
inst main()
{
        int a;
        printf("Hello\n");
        return 0;
}
```

$cc sample1.c …..will ignore warnings
…...You can proceed to execute

//To find the hidden bug, do the following
$cc -Wall -Wextra sample1.c …….-Wall (useful warnings), -Wextra (additional warnings)
Sample1.c:8:5: warning: unused variable 'a'
Identity: filename - test.c line number - 8, types of message - warning, remaining text - actual message.